

AGROVISION OLMOS

Documentación del sistema AGV-MI

De qué trata

Aplicación web para cargar reportes Excel de cartillas de calidad (MP, PT y Plagas), validarlos con reglas configurables y mostrar errores en pantalla. Cubre uva, arándano, espárrago y palta.

Para qué sirve

Ahorrar revisión manual de reportes, centralizar las reglas de negocio en archivos de configuración y permitir que un cambio de validación se haga sin reescribir toda la interfaz.

Dónde usarlo

El sistema está publicado en Netlify. Entra desde el navegador:
<https://aseguramiento-de-la-calidad.netlify.app/#/inicio>

Código fuente: <https://github.com/qaagrovision-design/QA-MI>

Cómo funciona este sistema

AGV-MI es el sistema web de Manejo de la Información de Agrovisión Olmos. Sirve para analizar cartillas Excel de calidad agrícola sin revisar celda por celda a mano.

De qué trata

Recibe reportes de Materia Prima (MP), Producto Terminado (PT) y Plagas para uva, arándano, espárrago y palta. Cada reporte se compara con reglas de negocio guardadas en archivos de configuración.

Para qué sirve

- Detectar campos vacíos, fuera de rango o con valor incorrecto.
- Ver el error en la tabla con un mensaje claro.
- Exportar Excel para corregir o entregar el resultado.
- Limpiar el módulo y cargar otro archivo cuando termine.
- Cambiar una regla editando el archivo de configuración, sin rehacer la pantalla.

Dónde está publicado (Netlify)

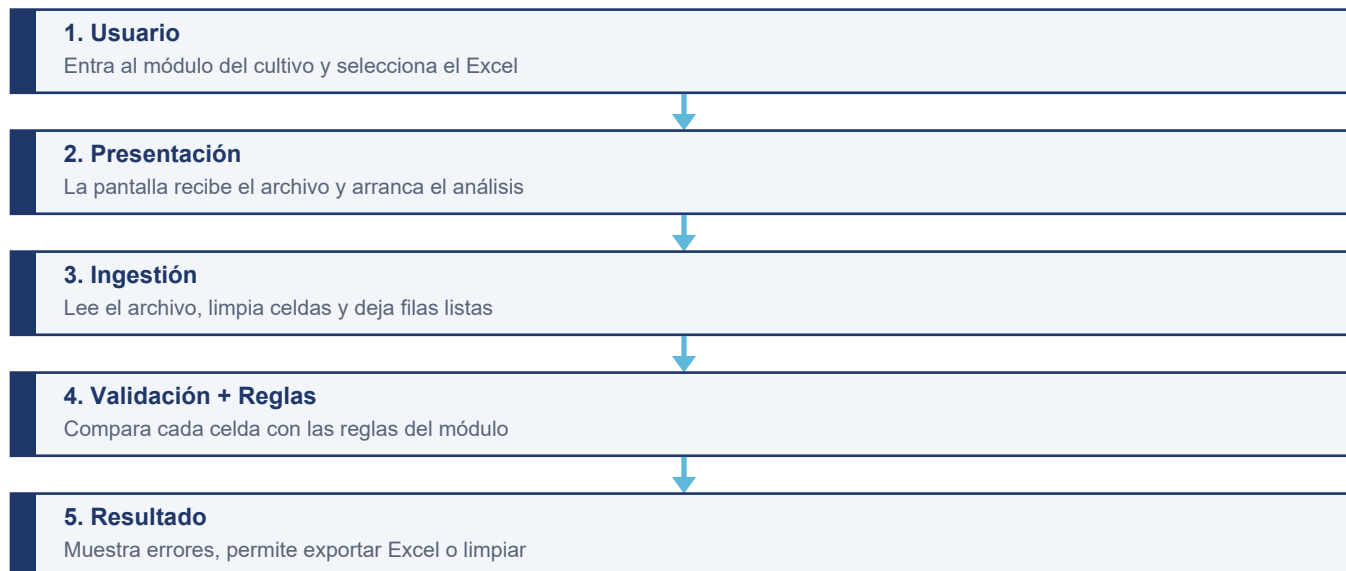
El sistema está alojado en Netlify. Netlify es el servicio que publica la web en internet y la actualiza cuando se sube código a GitHub.

Enlace del sistema (pantalla de inicio):

<https://aseguramiento-de-la-calidad.netlify.app/#!/inicio>

Cualquier persona con el enlace lo abre en el navegador. No necesita instalar el proyecto para usarlo.

Flujo completo (con conectores)



Las 4 capas y cómo se conectan



Las flechas muestran el paso de datos. Las reglas alimentan la validación: el archivo de configuración no se ejecuta solo; engine lo lee y lo aplica.

1. Usabilidad: botones que usa la persona en pantalla

Esta parte es para quien opera el sistema día a día (no para programar). Después de cargar un Excel verá acciones como Limpiar y Exportar.

Limpiar

Sirve para vaciar el módulo y empezar de nuevo. Borra el archivo cargado, la tabla y los errores de esa sesión. Úselo cuando termine con un reporte o antes de subir otro Excel distinto.

- No borra las reglas del sistema.
- No borra nada en GitHub ni en Netlify.
- Solo reinicia lo que tiene abierto en esa pantalla.

Exportar Excel (filtrado)

Descarga un Excel con las inspecciones de la fecha que está revisando, en el orden/formato que usa el área. Sirve para trabajar solo con lo del día, sin cargar todo el archivo original otra vez.

- Útil para entregar o revisar un corte por fecha.
- Si hay varias cartillas cargadas, puede elegir una o todas.

Exportar Excel con errores resaltados

Descarga el Excel completo (o el de la cartilla) marcando las filas o celdas con error. Sirve para corregir en Excel y volver a cargar, o para mostrar a alguien qué falló.

- El resaltado ayuda a ubicar el problema rápido.
- Es el export de usabilidad para corrección, no solo un archivo vacío.

Tabla, filtros y celdas en error

Tras revisar una fecha, la tabla muestra las filas con problema. Las celdas vacías obligatorias o con valor incorrecto se pintan para ubicar el fallo. Puede filtrar por cliente/formato según el módulo y buscar por Id o Lote.

- Celda pintada = hay que corregir ese dato en el Excel.
- El mensaje al pasar el mouse (o título) indica qué falló.
- Limpiar reinicia la pantalla para cargar otro archivo.

Archivos de reglas (configuración)

Las reglas no están “escondidas” en la pantalla: están en archivos de texto tipo JSON dentro de rules/modulos/. JSON es solo el formato del archivo (clave y valor). Quien mantiene el sistema edita esos archivos para cambiar obligatoriedad, rangos o mensajes.

La persona que solo usa la web no necesita tocar JSON: carga Excel, ve errores, exporta y limpia.

Acción	Para qué la usa la persona
Limpiar	Reinicia y permite cargar otro Excel
Excel filtrado	Baja lo de la fecha / cartilla en revisión

Acción	Para qué la usa la persona
Excel con errores	Baja el archivo marcando fallas
JSON / reglas	Solo mantenimiento; no lo toca el usuario diario

2. Código, editor y publicación

Código fuente en GitHub

El código del proyecto está en este repositorio:

```
https://github.com/qaagrovision-design/QA-MI
```

En el repositorio están presentation, ingestion, engine y rules, más index.html y netlify.toml. Para trabajar se clona y se abre en Visual Studio Code.

Cómo se abre (Visual Studio Code)

- Instalar Visual Studio Code.
- Clonar o abrir la carpeta del proyecto QA-MI.
- Abrir la terminal integrada.
- Editar pantallas, reglas o estilos según el cambio.

Cómo subir una versión nueva

Después de editar en Visual Studio Code:

```
git status
git add .
git commit -m "descripcion corta del cambio"
git push origin main
```

Comando	Para qué
git status	Ver qué archivos cambiaron
git add .	Preparar los cambios
git commit -m "..."	Guardar la versión con un mensaje
git push origin main	Subir la versión a GitHub

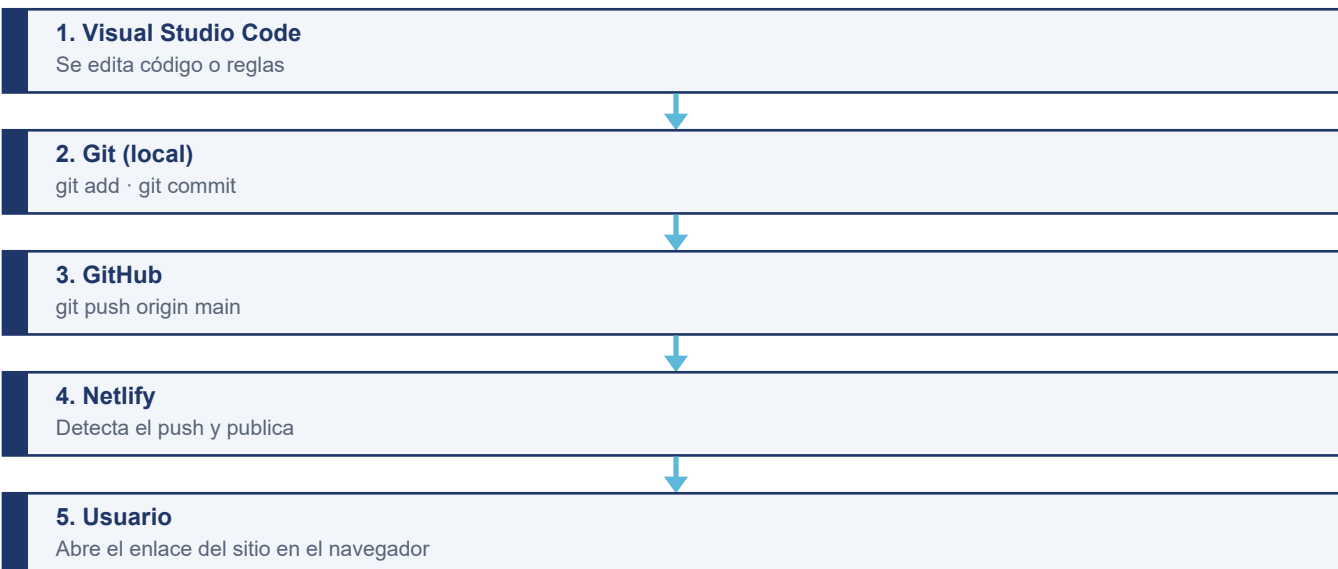
Netlify y el sitio público

Netlify está conectado a GitHub. Cuando hay un push a la rama main, Netlify hace el build y actualiza el sitio automáticamente.

Sitio publicado (inicio del sistema):

```
https://aseguramiento-de-la-calidad.netlify.app/#!/inicio
```

Flujo de publicación (con conectores)



3. Arquitectura CDA

Se usa Configuration Driven Architecture: las reglas de negocio viven en archivos de configuración y el código solo las interpreta.

Capa	Carpeta	Qué hace
Presentación	presentation/	Pantallas, rutas, tablas, export
Ingestión	ingestion/	Abrir Excel y normalizar datos
Validación	engine/	Aplicar las reglas sobre las filas
Reglas	rules/modulos/	Definir validaciones en configuración

4. Mapa del repositorio

Carpeta / archivo	Contenido
index.html	Entrada de la aplicación
presentation/	HTML, CSS, JS, idiomas, catálogos
presentation/js/config/	Rutas y configuración general
presentation/js/modules/	Lógica por pantalla
ingestion/	Lectura y normalización del reporte
engine/	Interpretación y evaluación de reglas
rules/modulos/	Reglas de negocio por cultivo/módulo
netlify.toml	Configuración de publicación en Netlify

5. Módulos y rutas

Ruta	Función
#/inicio	Indicadores y resumen operativo
#/dashboard	Catálogo de variedades
#/trazabilidad	Consulta de trazabilidad
#/cartillas	Listado de cartillas

Cultivo	MP	PT	Plagas
Uva	#/uva/mp	#/uva/pt	#/uva/plagas
Arándano	#/arandano/mp	#/arandano/pt	#/arandano/plagas
Espárrago	#/esparrago/mp	#/esparrago/pt	#/esparrago/plagas
Palta	#/palta/mp	#/palta/pt	#/palta/plagas

6. Qué archivo editar

Si necesita...	Edite esto	Evite tocar
Obligatorio, rango, mensaje, valor exacto	rules/modulos/*.rules.json	engine/ y HTML
Textos de pantalla o idioma	presentation/i18n/*.json	rules/
Nueva ruta o menú	routes.config.js y sidebar config	engine/
Diseño o tabla de un módulo	views + css + *.service.js	rules/ (si no cambia validación)
Cómo se lee el Excel	ingestion/	mensajes de reglas de calidad
Nuevo tipo de validación	engine/	solo si la configuración ya no alcanza

7. Propiedades de reglas (con ejemplo)

Solo aparecen propiedades que sí se usan en rules/modulos/. Cada una trae significado, ejemplo y lectura en simple.

es-obligatorio

Qué hace: La celda no puede quedar vacía.

Ejemplo real

```
{
  "numero": 11,
  "nombre-de-la-columna": "Cant Muestra",
  "es-obligatorio": true
}
```

En simple: Si Cant Muestra viene vacío → error.

si-falla-mostrar

Qué hace: Texto que ve el usuario cuando esa regla falla.

Ejemplo real

```
{
  "numero": 45,
  "nombre-de-la-columna": "Fruto Rojo",
  "es-obligatorio": true,
  "si-falla-mostrar": "Fruto Rojo es obligatorio"
}
```

En simple: En pantalla aparece exactamente ese mensaje.

igual-a-valor

Qué hace: La celda debe ser exactamente ese valor.

Ejemplo real

```
{
  "numero": 62,
  "nombre-de-la-columna": "Transporte Protegido",
  "es-obligatorio": true,
  "igual-a-valor": "CUMPLE",
  "si-falla-mostrar": "Debe ser CUMPLE"
}
```

En simple: Solo pasa si escribe CUMPLE. Otro texto → error.

mínimo / máximo

Qué hace: La celda debe ser un número dentro del rango.

Ejemplo real

```
{
  "numero": 61,
  "nombre-de-la-columna": "T° PULPA",
  "es-obligatorio": true,
  "minimo": 15,
  "maximo": 38.5,
  "si-falla-mostrar": "T pulpa entre 15 y 38.5"
}
```

En simple: 14.9 falla. 20 pasa. 40 falla.

longitud-exacta

Qué hace: La celda debe tener exactamente N caracteres.

Ejemplo real

```
{
  "numero": 10,
  "nombre-de-la-columna": "Lote",
  "es-obligatorio": true,
  "longitud-exacta": 10,
  "si-falla-mostrar": "Lote debe tener 10 caracteres"
}
```

En simple: Un lote de 9 o 11 caracteres falla.

validar-solo-si-tiene-valor

Qué hace: Si está vacío no valida; si tiene algo, sí aplica las demás reglas.

Ejemplo real

```
{
  "numero": 35,
  "nombre-de-la-columna": "Daño por plaga/Enfermedad",
  "validar-solo-si-tiene-valor": true
}
```

En simple: Vacío = ok. Si ponen un valor inválido, ahí sí marca error.

debe-estar-vacio

Qué hace: Esa columna no debe traer dato (en esa cartilla).

Ejemplo real

```
{
  "numero": 23,
  "nombre-de-la-columna": "Calibre",
  "debe-estar-vacio": true,
  "si-falla-mostrar": "Debe estar vacio"
}
```

En simple: Si Calibre viene lleno en MPBAR → error.

no-puede-repetirse + agrupar-por-columna

Qué hace: No permite duplicados. Con agrupar-por-columna, el duplicado se mira dentro del mismo grupo (ej. misma fecha).

Ejemplo real

```
{
  "numero": 10,
  "nombre-de-la-columna": "Lote",
  "es-obligatorio": true,
  "longitud-exacta": 10,
  "no-puede-repetirse": true,
  "agrupar-por-columna": 72,
  "si-falla-mostrar": "Lote único por fecha"
}
```

En simple: El mismo lote dos veces en la misma fecha → error.

igual-a-columna

Qué hace: El valor debe ser igual al de otra columna del mismo registro.

Ejemplo real

```
{
  "numero": 20,
  "nombre-de-la-columna": "Fecha Cosecha",
  "es-obligatorio": true,
  "igual-a-columna": 82,
  "si-falla-mostrar": "Fecha cosecha debe igualar fecha de inspección"
}
```

En simple: Columna 20 debe coincidir con la columna 82.

valores-permitidos

Qué hace: Solo acepta valores de una lista cerrada.

Ejemplo real

```
{
  "numero": 12,
  "nombre-de-la-columna": "Med Muestra",
  "es-obligatorio": true,
  "valores-permitidos": ["UNIDADES"],
  "si-falla-mostrar": "Med. Muestra debe ser UNIDADES"
}
```

En simple: Si pone GRAMOS u otro texto → error. Solo UNIDADES pasa.

patron-regex

Qué hace: La celda debe cumplir un formato (máscara de texto).

Ejemplo real

```
{
  "numero": 14,
  "nombre-de-la-columna": "Guía de Remisión",
  "es-obligatorio": true,
  "patron-regex": "^TG\\d{2}-\\d{8}$",
  "si-falla-mostrar": "Debe ser TG##-##### (ej: TG16-00002196)"
}
```

En simple: TG16-00002196 pasa. TG-123 o ABC falla.

Archivos de reglas por cultivo

Módulo	Archivo en rules/modulos/
Uva MP / PT / Plagas	uva-mp · uva-pt · uva-plagas
Arándano MP	arandano-mp-mpbar · mpgar · mpha
Arándano PT	arandano-pt-ptbpar · ptlpar · pthpar
Arándano Plagas	arandano-plagas
Espárrago	esparrago-mp-mpes · esparrago-pt · esparrago-plagas
Palta	palta-mp · palta-pt · palta-plagas

8. Detalle de cada capa

Validación (engine/)

Archivo	Rol
rule-engine.js	Carga las reglas y analiza el reporte
cartilla-rules.adapter.js	Adapta reglas al formato de la UI
cartilla-cell-validation.js	Valida celda por celda
pt-rule-validators.js	Validadores extra de PT

Ingestión (ingestion/)

Lee el archivo y deja datos uniformes. No decide si un defecto está mal: eso lo hace la validación con las reglas.

Archivo	Rol
report-ingestor.js	Orquesta la ingestión
file-reader.js	Abre el archivo en el navegador
sheet-matrix.js	Hoja → matriz
cell-reader.js	Normaliza valor de celda
row-extractor.js	Saca encabezados y filas
archivo-validator.js	Revisa estructura básica

Presentación (presentation/)

```
views/cultivos/{cultivo}-{tipo}.html
js/modules/cultivos/{cultivo}-{tipo}/
  *.module.js → arranque de la ruta
  *.service.js → carga archivo, valida, pinta
  *.config.js → rutas a rules / data
css/pages/{cultivo}-{tipo}.css
```

9. Caso guiado

Ubicar módulo

Ejemplo: Arándano MP, cartilla MPBAR



Abrir rules

rules/modulos/arandano-mp-mpbar.rules.json



Editar columna

Buscar Lote; poner longitud-exacta: 10



Probar

Abrir el sitio, subir Excel y verificar el error



Publicar

git push → Netlify actualiza el enlace público

10. Prácticas y límites

- Quién usa la web: cargar, ver errores, exportar, limpiar.
- Quién mantiene: editar rules/ y subir a GitHub.
- Probar con un Excel real de la cartilla afectada.
- Para publicar: push a main; Netlify actualiza solo.
- Insertar columna al final del Excel es más seguro que al medio.

11. Glosario

Término	Significado
CDA	Arquitectura guiada por configuración
MP / PT / Plagas	Tipos de cartilla
Reglas / JSON	Solo mantenimiento; no lo toca el usuario diario

Término	Significado
Limpiar	Reinicia y permite cargar otro Excel
Excel filtrado	Baja lo de la fecha / cartilla
Excel con errores	Baja el archivo marcando fallas
Usabilidad	Botones y tabla que usa la persona en pantalla
GitHub	Repositorio del código
Netlify	Publica el sitio web desde GitHub
Push	Subir una versión nueva a GitHub
Sitio	https://aseguramiento-de-la-calidad.netlify.app/#/inicio

Sitio: <https://aseguramiento-de-la-calidad.netlify.app/#/inicio>

Código: <https://github.com/qaagrovision-design/QA-MI>

Flujo: Visual Studio Code → git push → GitHub → Netlify actualiza la web.